

# GPU-Accelerated Pareto Frontier Enumeration for the Multi-Objective Traveling Salesperson Problem via Dynamic Layer Cutset on MDDs

## 1 Methodology

We describe the CPU baseline algorithm and the GPU-accelerated variant for computing the exact Pareto frontier of a multi-objective Traveling Salesperson Problem (TSP) using Multi-valued Decision Diagrams (MDDs) and the dynamic layer cutset method.

### 1.1 Problem Setting

Given an MDD  $\mathcal{M}$  with  $L$  layers encoding all feasible TSP tours, each arc carries a weight vector  $\mathbf{w} \in \mathbb{Z}^p$  ( $p$  objectives). A root-to-terminal path corresponds to a tour with objective vector equal to the sum of its arc weights. The goal is to enumerate the *Pareto frontier*—the set of all non-dominated objective vectors—over all paths.

### 1.2 CPU Baseline: Dynamic Layer Cutset

The CPU algorithm proceeds in three phases.

#### 1.2.1 Phase 1: Bidirectional Expansion

Two sets of partial Pareto frontiers are grown from both ends of the MDD:

- **Top-down:** Starting from the root node (layer 0) with the singleton frontier  $\{\mathbf{0}\}$ , expand layer by layer. At layer  $\ell$ , for each node  $v$  and each incoming arc  $(u \rightarrow v)$  with weight  $\mathbf{w}$ , every point  $\mathbf{x}$  in  $u$ 's frontier generates a candidate  $\mathbf{x} + \mathbf{w}$ . Candidates arriving at the same node are merged and dominance-filtered to retain only non-dominated points.
- **Bottom-up:** Symmetric expansion from the terminal node (layer  $L-1$ ) towards the root, using outgoing arcs in reverse.

A *layer-value heuristic*  $V(\ell) = \sum_{v \in \text{layer}(\ell)} |\text{frontier}(v)| \cdot |\text{arcs}(v)|$  estimates the expansion cost of the next layer. At each step, the direction with the smaller layer value is expanded. The two fronts meet at a *cutset layer*  $c$  chosen dynamically to minimise the total work.

#### 1.2.2 Phase 2: Cutset Convolution

At the cutset layer, each node  $v$  has a top-down frontier  $F_v^\downarrow$  and a bottom-up frontier  $F_v^\uparrow$ . The complete Pareto frontier is contained in

$$\bigcup_v \{\mathbf{a} + \mathbf{b} : \mathbf{a} \in F_v^\downarrow, \mathbf{b} \in F_v^\uparrow\}.$$

The CPU processes nodes one-by-one: for each node  $v$ , it forms all  $|F_v^\downarrow| \times |F_v^\uparrow|$  Cartesian-product sums and incrementally merges them into a running global frontier, checking each new candidate against the current frontier for dominance. The per-candidate cost is  $O(|F|)$  where  $|F|$  is the current frontier size, giving total complexity  $O(\sum_v |F_v^\downarrow| \cdot |F_v^\uparrow| \cdot |F|)$ .

### 1.2.3 Phase 3: Result

The global frontier after processing all cutset nodes is the exact Pareto frontier.

## 1.3 GPU-Accelerated Top-Down Variant

We first discuss a purely top-down GPU-accelerated enumeration algorithm. The key idea is to keep Pareto frontier data on the GPU throughout the entire process, minimising host-device transfers.

### 1.3.1 Phase 1: GPU Single-Direction Expansion

**Data packing.** All MDD layers are packed into flat CSR-style arrays on the host and transferred to the GPU once at startup. For each layer  $\ell$ , we store the incoming-arc offsets, source-node indices, and arc weights.

**Expansion kernels.** Each layer expansion is a three-step GPU pipeline:

1. **Candidate generation** (`expand_candidates_points_kernel`): Each warp processes one edge, reading source-node frontier points and adding the arc weight vector. This is embarrassingly parallel—every candidate is independent.
2. **Per-node dominance filtering**: Candidates arriving at the same destination node are compared pairwise to filter out dominated points. We developed two versions of this kernel:
  - **v1 (Dense 2D Grid, `mark_dominated_by_dst_tiled_kernel`)**: A 2D grid assigns one block-row per node, launching `<<<num_nodes, max.blocks>>>`. Each thread loads its candidate into registers, then iterates over shared-memory tiles of all candidates in the same node. However, highly unbalanced MDD layers result in massive block scheduling overhead, as many nodes with zero candidates still force the GPU to schedule empty blocks.
  - **v2 (Load-Balanced 1D Grid, `mark_dominated_1d_kernel`)**: To solve the imbalance, we compute `ceil_div(node_candidates, threads)` for each node and perform a parallel prefix sum to find the exact total number of required blocks. The kernel is launched with a 1D grid of exactly this size, completely eliminating empty block scheduling. Within the kernel, a custom binary search on the prefix-sum array maps each 1D block to its correct destination node. This perfect load balancing drastically reduces CUDA launch overhead, cutting total expansion time by over 30%.
3. **Stream compaction** (`scatter_alive_kernel`): An exclusive prefix sum on the alive flags gives output indices; surviving points are scattered into a compact output buffer.

**Parallelism advantage.** On the CPU, expansion is sequential over nodes and arcs. On the GPU, all edges in a layer are expanded simultaneously (step 1), and dominance filtering for all nodes in the layer runs in parallel (step 2, one block-row per node). For the largest layers (24 024 nodes, 168 168 edges, 4M+ points), this yields  $>100\times$  speedup over the CPU expansion alone.

### 1.3.2 Phase 2: Result

The global frontier at the terminal node is returned directly from device memory.

## 1.4 GPU-Accelerated Coupled Variant

Building upon the highly optimized GPU expansion kernels described above, we also ported the dynamic layer cutset (coupled) method to the GPU.

### 1.4.1 Phase 1: GPU Bidirectional Expansion

Data packing now encompasses both incoming arcs (for top-down) and outgoing arcs (for bottom-up). The layer expansion logic directly references the *Expansion kernels* from the GPU Top-Down Variant.

**Layer-value heuristic.** The heuristic  $V(\ell)$  is computed on the GPU using a `thrust::reduce`, returning a single scalar to the host. This avoids copying the entire frontier back for the selection decision.

### 1.4.2 Phase 2: GPU Batched Convolution

The cutset convolution is the computational bottleneck for the coupled algorithm. The CPU processes nodes sequentially; the GPU version uses a *batched frontier-filter pipeline*:

1. **Batching.** Cutset nodes are sorted by decreasing product size and grouped into batches of up to 2M Cartesian products each.
2. **Cartesian product** (`cartesian_product_kernel`): One CUDA block per node computes all  $|F_v^\downarrow| \times |F_v^\uparrow|$  sums. Threads stride over the product space.
3. **Frontier filter** (`mark_dominated_by_frontier_kernel`): Each candidate is checked against the *existing* frontier using shared-memory tiling. Cost:  $O(\text{batch\_size} \times |F|)$ . This is the key efficiency gain: with  $|F| \approx 5\text{--}6\text{K}$  and batch size  $\approx 2\text{M}$ , each thread checks  $\sim 6\text{K}$  frontier points loaded tile-by-tile into shared memory. Since frontier points are loaded cooperatively into shared memory and reused by all 128 threads in a block, the effective memory bandwidth is amplified  $128\times$ .

This step typically eliminates  $>99.9\%$  of candidates (e.g.,  $2\text{M} \rightarrow \sim 200\text{--}2000$  survivors).

4. **Frontier update** (`mark_frontier_dominated_kernel`): Existing frontier points dominated by new survivors are removed. Cost:  $O(|F| \times |\text{survivors}|)$ . Survivors are appended to the frontier.
5. **Final global dominance** (`mark_dominated_global_kernel`): After all batches, a single  $O(|F|^2)$  pass on the accumulated frontier ( $\sim 10\text{K}$  points) cleans up any cross-batch dominated points. This takes  $<2\text{ms}$  on the GPU.

**Parallelism advantage.** The CPU’s serial `convolute` checks each of the 303M candidate products against the frontier one at a time. The GPU checks 2M candidates *simultaneously*, with each of the  $\lceil 2\text{M}/128 \rceil \approx 15\text{K}$  thread blocks independently scanning the  $\sim 5\text{K}$  frontier via shared memory. The frontier filter is thus the GPU analogue of the CPU’s incremental merge, but with  $\sim 15\text{K}$ -way parallelism.

## 2 Results

We compare the CPU and GPU variants on two TSP instances with 15 cities, using the dynamic layer cutset method (method 3). All experiments were run on the same machine with a single NVIDIA GPU.

### 2.1 Summary

| Instance  | Objs | CPU Sols | CPU Time (s) | CUDA Time (s) | Speedup |
|-----------|------|----------|--------------|---------------|---------|
| seed108   | 3    | 831      | 2.78         | 5.37          | 0.52×   |
| seed11417 | 4    | 5372     | 18.24        | 5.94          | 3.07×   |

Table 1: CPU vs. CUDA execution times on 15-city TSP instances. Solution counts match exactly in both cases.

### 2.2 Phase-Level Breakdown (4 Objectives)

| Phase           | CPU (s)    | CUDA (s) | Notes                         |
|-----------------|------------|----------|-------------------------------|
| MDD Packing     | —          | 0.14     | One-time host→device transfer |
| Expansion (TD)  | ~0.7       | 0.12     | 9 layers, up to 4M points     |
| Expansion (BU)  | (combined) | 0.02     | 7 layers                      |
| Convolution     | 17.5       | 5.29     | 153 batches of ~2M products   |
| Final dominance | —          | 0.001    | $O(10K^2)$ on GPU             |
| <b>Total</b>    | 18.24      | 5.94     |                               |

Table 2: Phase breakdown for the 4-objective, 15-city instance. The convolution phase accounts for 96% of CPU time but only 89% of CUDA time.

The GPU expansion phase is roughly  $6\times$  faster than CPU for the same work (0.14 s vs.  $\sim 0.7$  s), and the convolution phase achieves a  $3.3\times$  speedup (5.29 s vs. 17.5 s).

### 2.3 Why the 3-Objective Case is Slower on GPU

For the 3-objective instance, the GPU variant is  $\sim 2\times$  slower than the CPU. Three factors explain this:

1. **Small problem size.** The 3-objective instance has far fewer Pareto-optimal points at each MDD node (the Pareto front size scales super-polynomially in the number of objectives). The CPU finishes expansion and convolution in 2.78 s total. Most of this time is in the convolution, which on the CPU involves only moderate-size frontiers ( $\sim 800$  solutions).
2. **Fixed GPU overhead.** MDD packing (host→device transfer of all 17 layers) and initial CUDA context setup take  $\sim 0.8$  s regardless of problem difficulty. For a 2.78 s CPU run, this overhead alone consumes 29% of the CPU budget.
3. **Underutilised parallelism.** With only  $\sim 800$  final solutions and small intermediate frontiers, the GPU batched convolution cannot amortise kernel launch overhead. The frontier filter kernel processes 2M candidates against a frontier of  $\sim 800$  points, but 15K thread blocks

each scanning only  $\lceil 800/128 \rceil = 7$  shared-memory tiles finish very quickly, making kernel launch and synchronisation overhead a large fraction of wall time. In contrast, for the 4-objective case the frontier is  $\sim 5\text{--}6\text{K}$ , requiring  $\sim 47$  tiles per block, which keeps the GPU’s streaming multiprocessors occupied.

In summary, the GPU variant is designed for the regime where the convolution dominates wall time—which occurs with 4+ objectives, larger cities, or denser MDD structures. For small instances where the CPU finishes in a few seconds, the fixed overheads of GPU initialisation and data transfer outweigh the parallelism gains.